树与二叉树

树

树的定义

##

树是一种非线性的数据结构，它是由 n ($n \geq 0$) 个有限节点组成的一个具有层次关系的集合。

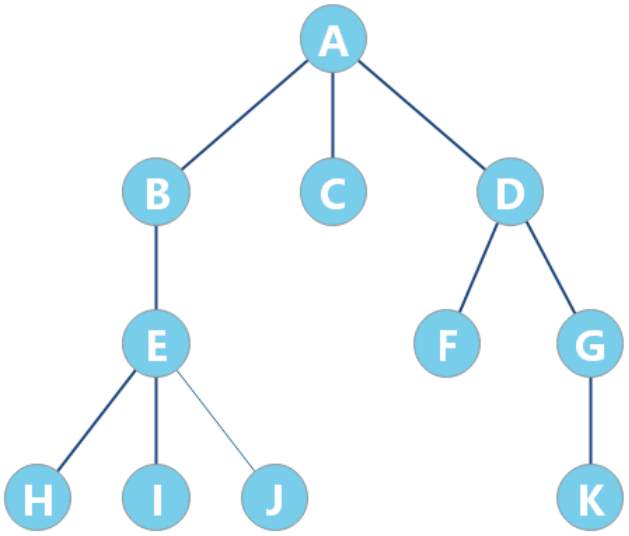
- 若$n = 0$，称为**空树**；
- 若$n > 0$，满足以下两个条件：
  - 有且仅有一个根节点（Root），没有父节点
  - 其余节点被分成 m（$m \geq 0$）个互不相交的集合，其中每一个集合本身又是一棵树，称为根的**子树**。

关键点：

1. 层次性： 树是“一对多”的关系。一个上层节点（父节点）可以连接多个下层节点（子节点）。
2. 无环性： 树中不能有回路。如果从一个节点出发，顺着连线走，绝对不能绕一圈回到原点。
3. 连通性： 树中的任意两个节点之间，有且仅有一条路径相连。

> 树就像现实中的树倒过来一样，由一个根发散出分支，分支再发散出叶子

树的基本术语



节点分类

1. 根节点 (Root)

- 定义： 树顶端唯一的那个节点，没有父节点。
- 图例： A

2. 叶子节点 (Leaf Node) / 终端节点

- 定义： 没有子节点（度为0）的节点。也就是树的“末梢”。
- 图例： C, H, I, J, F, K

3. 分支节点 / 内部节点

- 定义： 除了叶子节点以外的所有节点（度不为0）。
- 图例： B, D, E, G。

4. 父节点 / 双亲节点

- 定义： 一个节点若有下级节点，那它就是下级节点的父节点。
- 图例： B 是 E 的父节点；D 是 F 和 G 的父节点。

5. 子节点 / 孩子节点

- 定义： 一个节点含有的下级节点。
- 图例： B, C, D 是 A 的子节点；H, I, J 是 E 的子节点。

6. 兄弟节点

- 定义： 拥有同一个父节点的节点互称为兄弟。
- 图例： B, C, D 互为兄弟（父节点都是 A）；F, G 互为兄弟（父节点都是 D）。

7. 祖先节点

- 定义： 从根节点到该节点路径上的所有节点。
- 图例： H 的祖先是 E, B, A。

8. 子孙节点

- 定义： 某节点子树中的任一节点。
- 图例： B 的子孙包括 E, H, I, J。

9. 子树

- 定义： 树中的任意一个节点，加上它所有的后代节点，连起来就是一棵“子树”。
- 图例： 节点 D 以及它下面的 F, G, K 共同构成了根节点 A 的一颗子树。

数量与度量术语

1. 节点的度

- 定义： 一个节点拥有子树的个数
- 图例：
  - 节点 A 的度 = 3 (有 B, C, D)；
  - 节点 D 的度 = 2 (有 F, G)
  - 节点 C 的度 = 0

2. 树的度

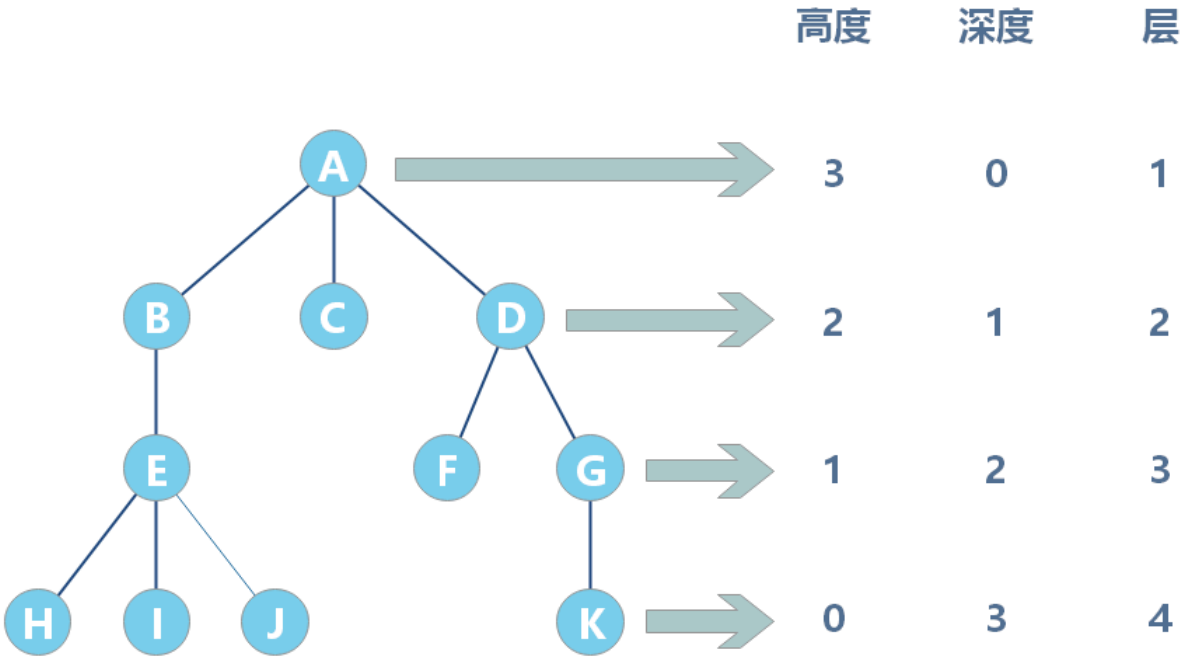
- 定义： 树中所有节点的度里面，最大的那个值。
- 图例： 最大的度是 3（来自 A 或 E），所以这棵树的度是 3。（这是一棵三叉树）。

3. 层次

- 定义： 从根开始算起，根为第 1 层，根的孩子为第 2 层，以此类推。
- 图例：
  - 第 1 层：A
  - 第 2 层：B, C, D
  - 第 3 层：E, F, G
  - 第 4 层：H, I, J, K

4. 树的深度/高度

- 定义： 树中节点的最大层次
- 图例： 这棵树的深度是 4。



5. 节点的层次

- 计算：节点的深度 + 1。
- 定义：通常规定根节点为第 1 层，从根往下数

6. 节点的高度

定义：节点到叶子节点的最长路径/历经的层数。高度是从下往上度量。

7. 节点的深度

定义：根节点到该节点的最长路径/历经的层数。深度是从上往下度量。

> 注意：在有的教材中，节点的深度 = 节点的层次；部分高阶算法书里，节点的深度=边的数量/路径长度

其他重要术语

1. 路径与路径长度

- 路径定义： 从一个节点到另一个节点经过的连线序列。
- 路径长度定义： 路径上经过的边的数目。
- 图例： A 到 H 的路径是 A -> B -> E -> H。
  - 涉及节点数：4个。
  - 路径长度：3

2. 森林

- 定义： $m\ (m \geq 0)$ 棵互不相交的树的集合。
- 图例：如果把根节点 A 去掉，剩下的部分就变成了包含三棵树（以 B, C, D 为根）的森林。

3. 有序树与无序树

- 有序树： 子节点之间是有左右顺序的，不能随意交换
- 无序树： 子节点之间没有顺序，交换了也不影响这棵树的含义。

树的性质

1. 节点数与边数的关系

树中的节点数等于边数加 1，公式如下：

$$n = e + 1$$

其中 n 是节点数，e 是边数

> 有 n 个节点，就会有 $n-1$ 条连线

2. 度数与节点数的关系

树中所有节点的度之和等于节点数减 1

$$\sum \text{Degree} = n - 1$$

3. 路径的唯一性

在树中，任意两个节点之间，有且仅有一条路径。

4. 树的度（高度）与节点的关系（针对 m 叉树）

度为 m 的树，第 i 层上至多有 m^{i-1} 个节点

树的分类

按“是否有序”分类

1. 无序树： 树中任意节点的子节点之间没有顺序。
2. 有序树：树中任意节点的子节点之间有严格的顺序（通常是从左到右）。

> 二叉树、B树等全都是有序树

按“度”分类

1. 普通树 / 多叉树：节点的度没有限制，可以有任意多个。
2. 二叉树：每个节点最多只能有 2 个子节点（度 ≤ 2）。

二叉树的“特殊形态”分类

1. 满二叉树：每一层都长满了，没有空缺。所有的叶子都在同一层。
2. 完全二叉树：节点必须从上到下、从左到右依次排列，不能“跳过”位置。

按“功能”分类

这些树通常基于二叉树，但加上了特殊的“规则”，用来解决特定问题（比如查找、排序、压缩）。

1. 二叉搜索树：左孩子 < 根节点 < 右孩子
2. 平衡二叉树：左右子树的高度差不能太大
3. 堆：根节点永远是最大（大顶堆）或最小（小顶堆）的
4. 哈夫曼树：带权路径长度最短

树的其他表示方法

1. 括号表示法

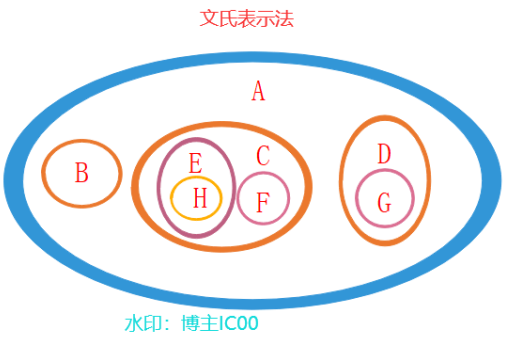
这是将树转化为线性字符串的最常用方法，非常适合在写代码时用来输入一棵树。

$$A(B(E(H,I,J)),C,D(F,G(K)))$$

2. 凹入表示法

```
A
|__ B
|    |__ E
|        |__ H
|        |__ I
|        |__ J
|__ C
|__ D
     |__ F
     |__ G
         |__ K
```

3. 集合/文氏图表示法



二叉树

二叉树中的每个节点最多有两个子节点，分别是左子节点和右子节点。

二叉树的定义与性质

定义

二叉树是 n $(n \geq 0)$ 个节点的有限集合。

1. 为空树（$n = 0$）。
2. 由一个根节点和两棵互不相交的、分别称为左子树和右子树的二叉树组成。

特点：

1. 度 ≤ 2：每个节点最多只能有 2 个孩子。绝不存在度为 3 的节点。
2. 有序性（左右有别）：二叉树是有序树。左孩子和右孩子是完全不同的。

> 即使根节点只有一个孩子，也要区分是“挂在左边”还是“挂在右边”。这在二叉树里是两种不同的树。

5 种基本形态

1. 空二叉树：连根都没有。
2. 只有一个根节点。
3. 根节点 + 左子树（右边空的）。
4. 根节点 + 右子树（左边空的）。
5. 根节点 + 左子树 + 右子树。

5 大性质

1. 每一层的节点数限制

- 定义：二叉树的第 i 层上至多有 2^{i-1} 个节点 ($i \geq 1$)
- 举例：第 1 层最多 1 个；第 2 层最多 2 个；第 3 层最多 4 个……

2. 整棵树的节点数限制

- 定义：深度为 k 的二叉树至多有 $2^k - 1$ 个节点 ($k \geq 1$)
- 原理：就是等比数列求和 ($1 + 2 + 4 + ... + 2^{k-1}$)。满二叉树就达到了这个最大值。

3. 叶子节点与度为2节点的关系

- 定义：对任何一棵二叉树，如果叶子节点数为 n_0，度为 2 的节点数为 n_2

$$n_0 = n_2 + 1$$

4. 完全二叉树的深度公式

- 定义：具有 n 个节点的完全二叉树，其深度为 $\lfloor \log_2 n \rfloor + 1$ （也就是 $\log_2 n$ 向下取整再加 1）。

5. 完全二叉树的编号规律

- 如果把节点从 1 开始编号（根为 1），对于任意节点 i：
  - 找爸爸： 它的父节点是 $\lfloor i/2 \rfloor$。
  - 找左孩子： 是 $2i$ （如果 $2i > n$，则无左孩子）。
  - 找右孩子： 是 $2i + 1$ （如果 $2i + 1 > n$，则无右孩子）。

二叉树常见术语

1. 左孩子

- 定义： 节点左边分支直接相连的那个子节点。
- 注意： 它是唯一的。

2. 右孩子

- 定义： 节点右边分支直接相连的那个子节点。
- 注意： 即使没有左孩子，右孩子依然叫“右孩子”

3. 左子树

- 定义： 以左孩子为根，往下延伸出来的整棵子树。

4. 右子树

- 定义： 以右孩子为根，往下延伸出来的整棵子树。

> 在二叉树中，左右是绝对不能颠倒的。如果把左子树移到右边，就变成了另一棵树。

5. 斜树

这是一种长歪了的树，本质上已经退化成了“链表”(一条线)。

- 左斜树： 所有节点都只有左孩子，一直往左歪。
- 右斜树： 所有节点都只有右孩子，一直往右歪。

二叉树的“存储结构

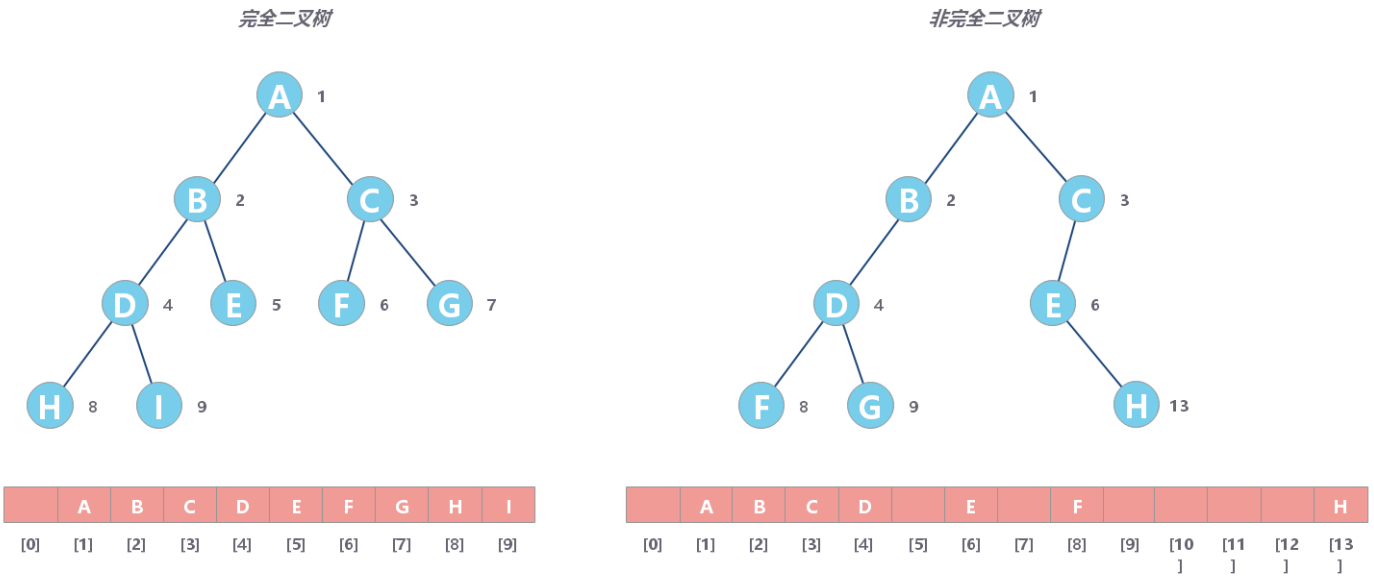
顺序存储结构

使用一个数组来存二叉树。 依靠完全二叉树的编号性质来确定谁是谁的孩子

怎么存：

假设有一个数组 `tree[]`，下标从 1 开始：

- 根节点存放在 `tree[1]`。
- 对于下标为 `i` 的节点：
  - 它的左孩子存在 `2 * i`。
  - 它的右孩子存在 `2 * i + 1`。



链式存储结构

像链表一样，每个节点不仅仅存数据，还存两个“指针”，分别指向左孩子和右孩子。

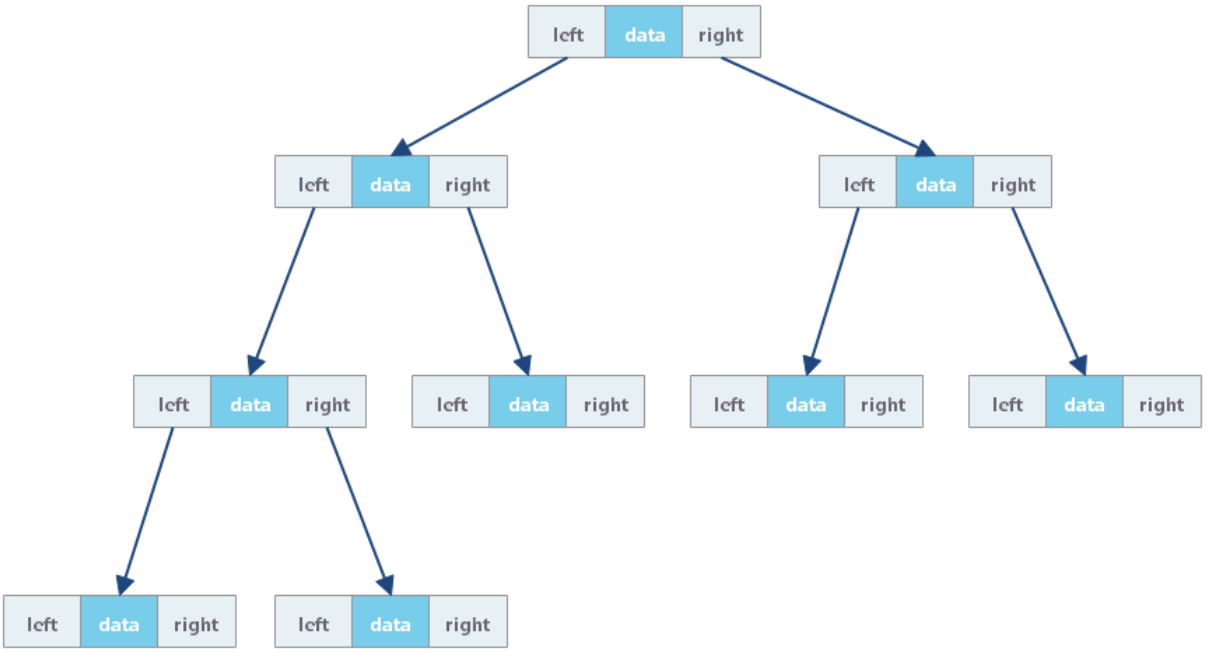
这种结构在数据结构中被称为**二叉链表**

1. 节点的内部结构

一个节点包含三个部分：

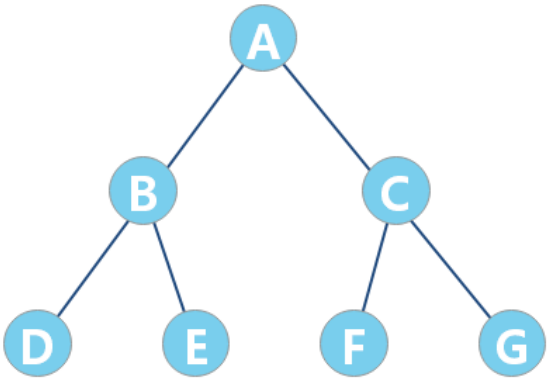
- 左指针域：存放指向左孩子的地址。
- 数据域：存放该节点真正的数据。

- 右指针域：存放指向右孩子的地址。



满二叉树

除了叶子节点之外，每个节点都有左右两个子节点，这种二叉树就叫作满二叉树

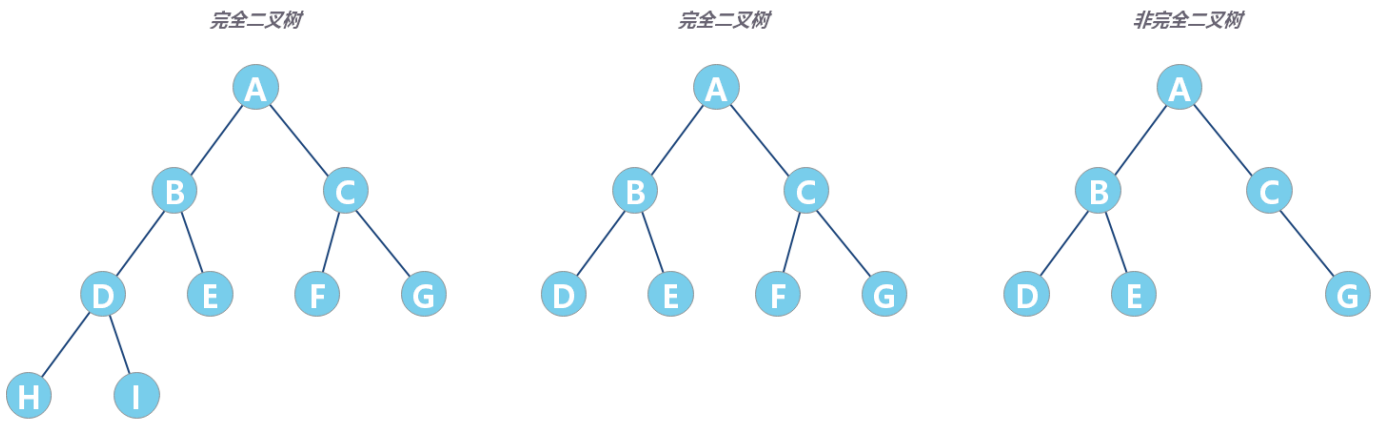


完全二叉树

> 从上到下，从左到右，绝不插队，绝不留空

特点：

1. 先把第一层填满。
2. 再把第二层填满。
3. 到了第三层（最后一层），必须紧贴着左边依次填，中间不能有空位



二叉树遍历

前序遍历

> 顺序：根-->左-->右

1. 根：先访问根节点。
2. 左：再把“左子树”当成一个新的树，去遍历它。
3. 右：最后把“右子树”当成一个新的树，去遍历它。

> 前序：根排在最前面访问。

```
      A
     /   \
    B     C
   / \   / \
  D   E F   G
```

- 前序遍历的目标是： 按照“根左右”的规则，把所有节点点名为一排。

> A -> B -> D -> E -> C -> F -> G

```cpp
#include <iostream>
using namespace std;

// 二叉树节点结构
struct TreeNode {
    int val;                // 节点值
    TreeNode* left;         // 左子节点
    TreeNode* right;        // 右子节点

    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 前序遍历：根节点 -> 左子树 -> 右子树
void preorderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;  // 空树直接返回
    }
```

```
    cout << root->val << " ";           // 1. 访问根节点
    preorderTraversal(root->left);      // 2. 遍历左子树
    preorderTraversal(root->right);     // 3. 遍历右子树
}

int main() {
    // 构建测试二叉树:
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5

    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "前序遍历结果: ";
    preorderTraversal(root);
    cout << endl;

    return 0;
}
```

中序遍历

> 左-->根-->右

1. 左：一定要先把左子树完完整整地走完。
2. 根：也就是中间。左边走不动了，才轮到自己（根节点）被访问。
3. 右：最后再去处理右子树。

```
        A
      /   \
     B     C
    / \   / \
   D   E F   G
```

- 中序遍历的目标是： 只要左边还有孩子，就不能打印自己，必须先去处理左边。

> D -> B -> E -> A -> F -> C -> G

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 中序遍历: 左子树 -> 根节点 -> 右子树
void inorderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;  // 空树直接返回
    }
```

```
    inorderTraversal(root->left);      // 1. 遍历左子树
    cout << root->val << " ";          // 2. 访问根节点
    inorderTraversal(root->right);     // 3. 遍历右子树
}

int main() {
    // 构建测试二叉树:
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5

    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "中序遍历结果: ";
    inorderTraversal(root);  // 预期输出: 4 2 5 1 3
    cout << endl;

    return 0;
}
```

后序遍历

> 左-->右-->根

1. 左：先去跑完左子树。
2. 右：再去跑完右子树。
3. 根：左右都搞定了，最后才访问根节点。

```
        A
      /   \
     B     C
    / \   / \
   D   E F   G
```

- 后序遍历的目标是： 只要下面还有孩子，就绝不打印自己。

> D -> E -> B -> F -> G -> C -> A

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 后序遍历: 左子树 -> 右子树 -> 根节点
void postorderTraversal(TreeNode* root) {
    if (root == NULL) {
        return;  // 空树直接返回
```

```
    }

    postorderTraversal(root->left);    // 1. 遍历左子树
    postorderTraversal(root->right);   // 2. 遍历右子树
    cout << root->val << " ";          // 3. 访问根节点
}

int main() {
    // 构建测试二叉树:
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5

    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "后序遍历结果: ";
    postorderTraversal(root);  // 预期输出: 4 5 2 3 1
    cout << endl;

    return 0;
}
```

层次遍历

层次遍历是二叉树的广度优先搜索遍历方式，核心规则是：按照二叉树的 “层次”（从根节点所在的第 1 层开始），以 “从上到下、同层节点从左到右” 的顺序，依次访问树中所有节点，直到所有层次的节点均被访问完毕。

- 前/中/后序：属于深度优先搜索 (DFS)
- 层次遍历：属于广度优先搜索 (BFS)

1. 遍历策略不同：层次遍历是 “横向拓展”，前 / 中 / 后序是 “纵向深入”（DFS）
2. 访问逻辑不同：层次遍历 “先扫完当前层，再推进到下一层”，DFS 则 “沿一条路径走到叶子节点，再回溯换路径”
3. 实现依赖不同：层次遍历常用 “队列” 存储待访问节点，DFS 常用 “栈” 或递归实现

> 从根节点开始，从上到下，每一层都从左到右访问

```
        A        <-- 第1层
      /   \
     B     C     <-- 第2层
    / \   / \
   D   E F   G   <-- 第3层
```

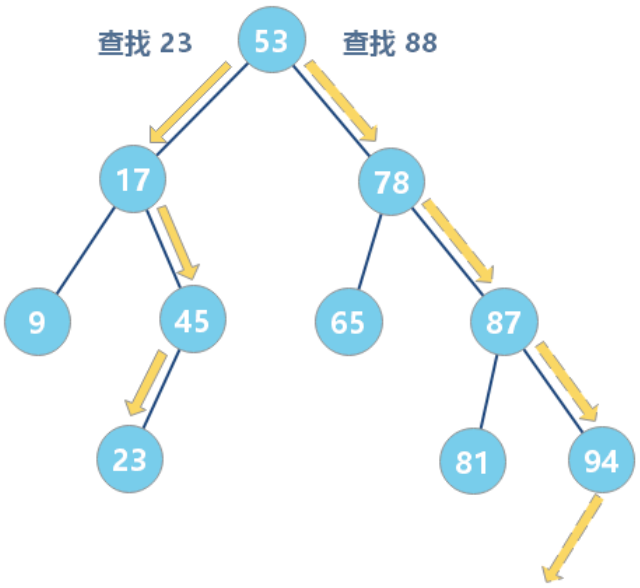
> A -> B -> C -> D -> E -> F -> G

```
#include <iostream>
#include <queue>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
```

```
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 层次遍历（广度优先遍历）
void levelOrder(TreeNode* root) {
    if (root == NULL) return;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* current = q.front();
        q.pop();

        cout << current->val << " ";  // 访问当前节点

        // 将左右子节点加入队列
        if (current->left != NULL) {
            q.push(current->left);
        }
        if (current->right != NULL) {
            q.push(current->right);
        }
    }
}

int main() {
    // 构建测试树:
    //       1
    //      / \
    //     2   3
    //    / \
    //   4   5
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);

    cout << "层次遍历结果: ";
    levelOrder(root);  // 预期输出: 1 2 3 4 5
    cout << endl;

    return 0;
}
```

二叉树的操作

查找



1. 在“普通二叉树”中查找 (无序)

这只是一棵普通的二叉树，节点里的数字是乱放的（没有左小右大的规律）

```
        8
      /   \
     3     9
    / \
   5   2
```

> 把每一个节点都翻一遍，直到找到为止

代码示例：基于前序遍历

```
// 在普通二叉树中查找目标值
TreeNode* searchNormalTree(TreeNode* root, int target) {
    if (root == NULL) {
        return NULL;  // 空树或到达叶子节点，未找到
    }

    // 先检查当前节点
    if (root->val == target) {
        return root;  // 找到目标，立即返回
    }

    // 在左子树中查找
    TreeNode* leftResult = searchNormalTree(root->left, target);
    if (leftResult != NULL) {
        return leftResult;  // 左子树找到，直接返回结果
    }

    // 左子树未找到，继续在右子树中查找
    return searchNormalTree(root->right, target);
}
```

2. 在“二叉查找树”中查找 (有序)

这才是二叉树查找的精髓。如果这棵树是 BST（左 < 根 < 右）。

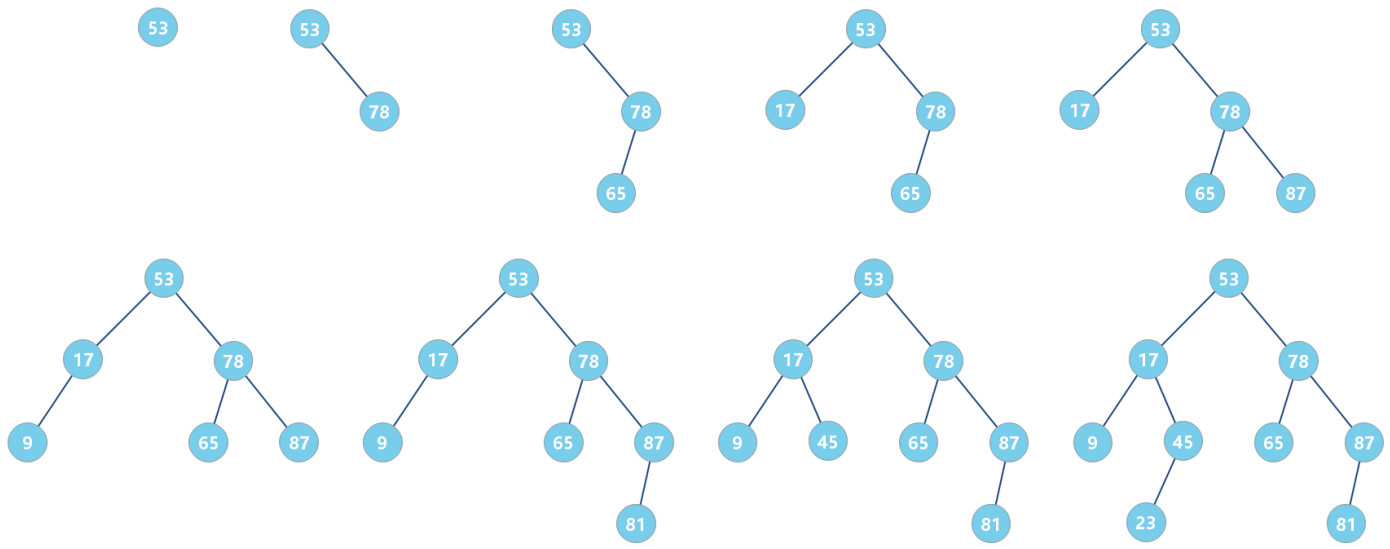
```
        10
      /    \
     5      15
    / \    /  \
   3   7  12  18
```

> 二分排除法

> 不需要遍历整棵树，只需要选一条路走到底。走了一条路，就意味着排除了另一半的树。

```cpp
// 在二叉搜索树(BST)中查找目标值
TreeNode* searchBST(TreeNode* root, int target) {
    if (root == NULL || root->val == target) {
        return root;  // 找到目标或到达空节点
    }

    // 利用BST特性：比当前值小则搜索左子树，大则搜索右子树
    if (target < root->val) {
        return searchBST(root->left, target);
    }

    return searchBST(root->right, target);
}
```

插入



二叉查找树建立
依次插入 53, 78, 65, 17, 87, 09, 81, 45, 23 节点

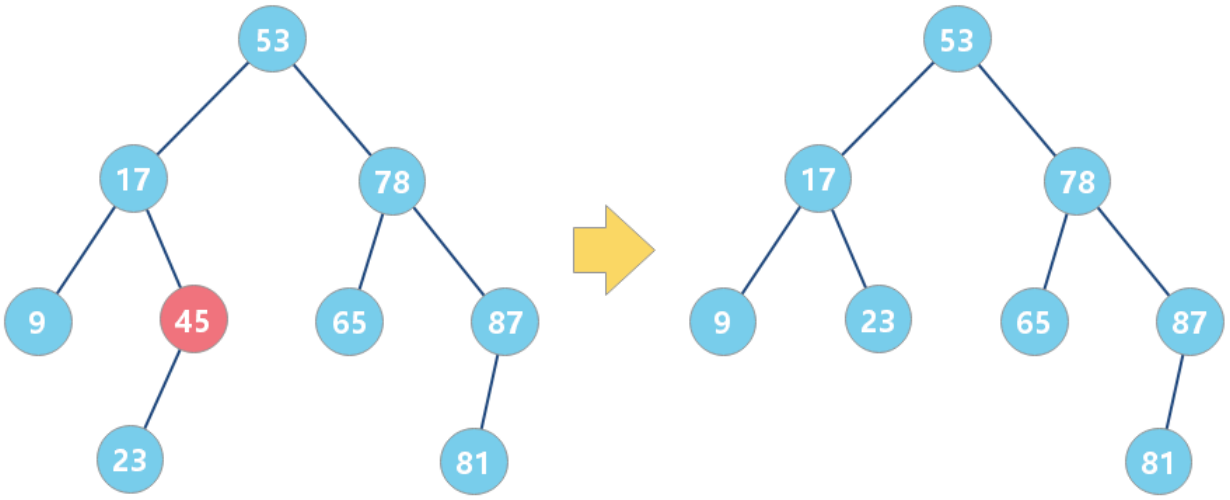
> 基于二叉搜索树（BST）的查找规则完成 “无效查找”，并将新节点挂载到查找终止的空位置。

1. 模拟查找流程
2. 定位插入位置：查找失败的终止空位置
3. 完成节点插入

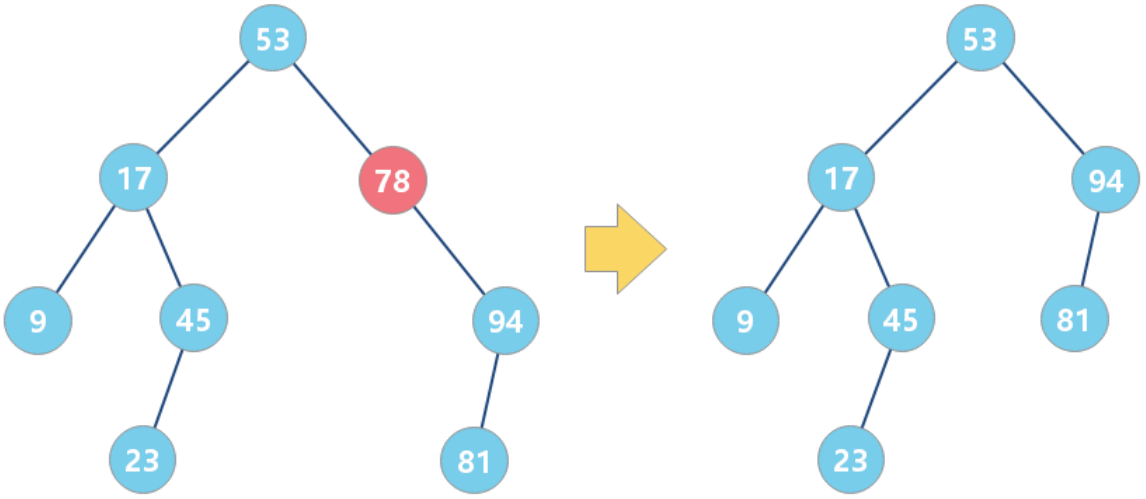
```cpp
#include <iostream>
```

```
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 在BST中插入新值
// 返回插入后的子树根节点
TreeNode* insertBST(TreeNode* root, int val) {
    // 找到插入位置，创建新节点
    if (root == NULL) {
        return new TreeNode(val);
    }

    // 利用BST特性确定插入方向
    if (val < root->val) {
        root->left = insertBST(root->left, val);
    } else if (val > root->val) {
        root->right = insertBST(root->right, val);
    }
    // 值已存在时不做处理

    return root;  // 返回当前节点以维持树结构
}

// 中序遍历验证BST结构
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

int main() {
    TreeNode* root = NULL;  // 初始为空树

    // 构建BST
    root = insertBST(root, 10);
    root = insertBST(root, 5);
    root = insertBST(root, 15);
    root = insertBST(root, 12);

    cout << "中序遍历结果: ";
    inorder(root);  // 预期输出: 5 10 12 15
    cout << endl;

    return 0;
}
```

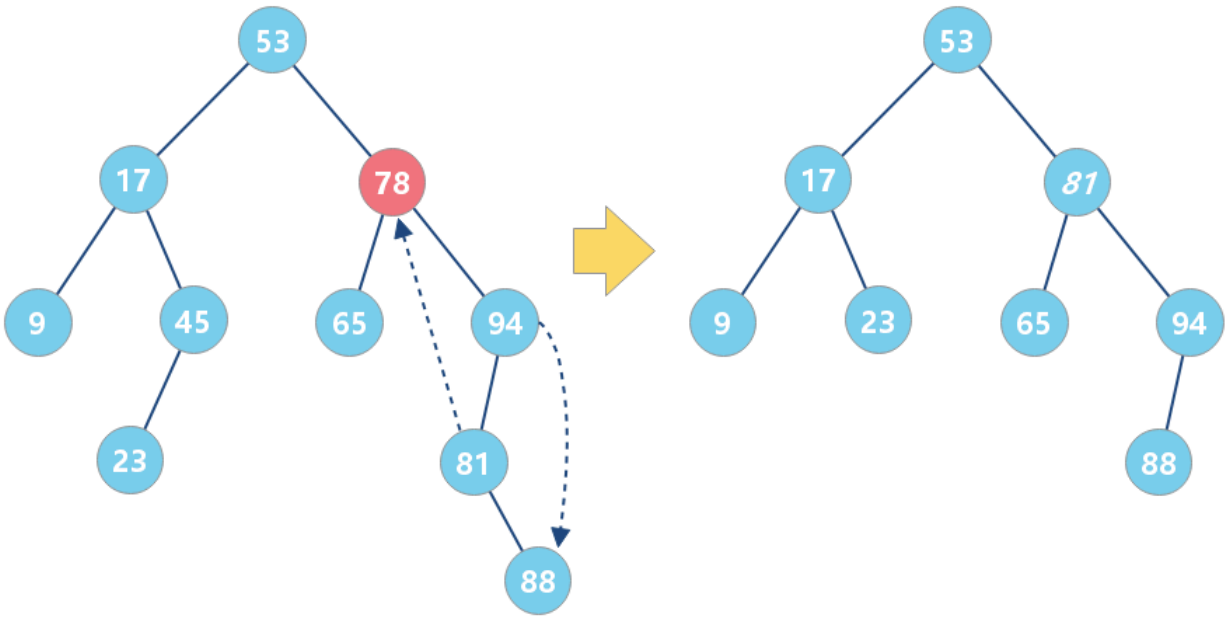
删除



二叉查找树
删除 45



二叉查找树
删除 78



思考：

- 删除一个节点后，必须保证剩下的树依然遵守“左 < 根 < 右”的规则，而且不能把树弄断了。
- 需要分 3 种情况 来处理。假设要删除节点 Target。

1. Target 是叶子节点

- 操作： 直接把它删掉，让它的父节点指向 `NULL` 即可
- 删除3节点

```
      10              10
     /  \            /  \
    5    15   ==>   5    15
   / \               \
  3*   7              7
```

2. Target 只有一个孩子

- 操作：直接让 Target 的孩子顶替 Target 的位置，连到 Target 的父节点上。
- 删除15节点

```
      10                10
     /  \              /  \
    5    15*   ==>    5     18
        /  \
     NULL   18
```

3. Target 有两个孩子

```
        10*
       /  \
      5    15
          /  \
         12   20
```

- 删除10 节点

- 问题： 不能简单地删掉 10，因为 5 和 15 谁来做老大？谁做老大的位置都可能破坏“左小右大”的规则。
- 策略： 需要找一个“替身”。 这个替身必须满足：比左边所有数都大，比右边所有数都小。
    - 右子树里的最小节点
    - 左子树里的最大节点

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

// 查找子树中的最小值节点
TreeNode* findMin(TreeNode* node) {
    while (node->left != NULL) {
        node = node->left;
    }
    return node;
}

// 从BST中删除指定值的节点
// 返回删除后的子树根节点
TreeNode* deleteNode(TreeNode* root, int key) {
    if (root == NULL) return NULL;  // 未找到目标节点

    // 递归查找目标节点
    if (key < root->val) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->val) {
        root->right = deleteNode(root->right, key);
    } else {
        // 找到目标节点，处理三种删除情况

        // 情况1: 无左子节点（包含无子节点的情况）
        if (root->left == NULL) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        }
        // 情况2: 无右子节点
        else if (root->right == NULL) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        }
        // 情况3: 有两个子节点
        else {
            // 找到右子树的最小节点作为替代
            TreeNode* temp = findMin(root->right);
            // 用替代节点的值替换当前节点值
            root->val = temp->val;
            // 删除右子树中的替代节点
            root->right = deleteNode(root->right, temp->val);
        }
    }
    return root;
}
```

二叉搜索树

二叉查找树（Binary Search Tree，简称 BST），也被称为二叉搜索树或二叉排序树

1. 左边都比我小： 若左子树不为空，则左子树上所有节点的值，都小于根节点的值。
2. 右边都比我大： 若右子树不为空，则右子树上所有节点的值，都大于根节点的值。
3. 递归有效： 它的左、右子树也分别为二叉查找树（也就是每一个局部都遵守上面两条规则）。

```
        10
      /    \
     5      15
    / \    /  \
   3   7  12  18
```

平衡二叉树

在计算机科学中，最经典的平衡二叉树叫 AVL 树

> 这棵树的任何一个节点，左边的高度和右边的高度，差值不能超过 1。

定义

- 平衡二叉树 (AVL) 必须满足两个条件：

1. 它首先得是一棵二叉查找树 (BST)：也就是满足“左 < 根 < 右”。
2. 高度平衡：任意节点的左子树和右子树的高度之差的绝对值不超过 1。

平衡因子

判断一棵树是不是“歪”了

$$BF = \text{左子树高度} - \text{右子树高度}$$

合格范围： -1, 0, 1。

- `0`：完美平衡（左右一样高）。
- `1`：左边比右边高 1 层（有点左倾，但还在忍受范围内）。
- `-1`：右边比左边高 1 层（有点右倾，但在忍受范围内）。

不合格： 只要绝对值 ≥ 2（比如 2, -2），说明失衡了，必须马上修整。

```
      5 (BF=0)
     / \
    3   7
```

- 5 的左高 1，右高 1。差 = 0。合格。

```
        10 (BF=1)
       /  \
      5    15
     /
    3
```

- 看 10： 左高 2 (5,3)，右高 1 (15)。差 = 1。合格。
- 看 5： 左高 1，右高 0。差 = 1。合格。

旋转

这是 AVL 树最核心、也是最难的机制。 当插入一个新节点导致 BF 变成 2 或 -2 时，树会自动“扭”一下。

1. 左旋
- 场景： 树往右边太长了。
- 操作： 逆时针转一下，把中间的节点提上去当根。
2. 右旋
- 场景： 树往左边太长了。
- 操作： 顺时针转一下，把左边的节点提上去。
3. 先左旋再右旋
- 场景： 也是左边重，但是是“左边的右孩子”重。
- 操作： 先把下半截扭直，再整体右旋。
4. 先右旋再左旋
- 场景： 右边重，但是是“右边的左孩子”重。

线索二叉树

- 定义

线索二叉树是对普通二叉树的优化结构——通过利用二叉树中 “空闲的空指针”（叶子节点的左 / 右孩子指针为 NULL，或非叶子节点的单侧空指

针），存储节点在某一遍历顺序下的 “前驱节点”和 “后继节点”引用，这些被改造后的空指针称为 “线索”，带线索的二叉树即为线索二叉树。

- 为什么需要线索二叉树?
1. 普通二叉树的痛点：遍历时需依赖栈或递归记录路径，才能找到节点的前驱 / 后继，效率低且占用额外空间；
2. 线索二叉树的优化：把原本闲置的空指针 “利用起来”，直接存前驱 / 后继信息，后续遍历无需栈 / 递归，可通过线索直接 “顺藤摸瓜” 访问所有节点，提升遍历效率。
- 关键概念：
1. 前驱节点：按指定遍历顺序，当前节点的 “前一个节点”；
2. 后继节点：按指定遍历顺序，当前节点的 “后一个节点”；
3. 线索：被改造后存储前驱 / 后继的空指针（比如叶子节点的左空指针存前驱，右空指针存后继）；
4. 标记位：每个节点需加两个标记位（LTag/RTag），区分当前左 / 右指针是 “指向孩子节点” 还是 “指向线索（前驱 / 后继）”。
- 如何区分指向节点还是线索
1. 节点结构改进

原来的节点：`[ lchild | data | rchild ]`

现在的节点：`[ lchild | ltag | data | rtag | rchild ]`

2. 标志位的含义

ltag = 0： `lchild` 指向左孩子。

ltag = 1： `lchild` 指向前驱节点。

rtag = 0： `rchild` 指向右孩子。

rtag = 1： `rchild` 指向后继节点。

- 中序线索二叉树

线索化是依赖于“遍历顺序”的。最常见的是基于中序遍历。

假设一棵树的中序遍历序列是： `D, B, E, A, F, C, G`

```
        A
      /   \
    B       C
   / \     / \
  D   E   F   G
```

1. 看节点 D

它的左边本来是 NULL。现在 `ltag=1`，左指针指向 `NULL`。

它的右边本来是 NULL。现在 `rtag=1`，右指针指向 B。

2. 看节点 E：

它的左边是 NULL。现在 `ltag=1`，左指针指向 B。

它的右边是 NULL。现在 `rtag=1`，右指针指向 A。

3. 看节点 F：

左边 `ltag=1`，指向 A。

右边 `rtag=1`，指向 C。

- C++ 结构体定义：

```cpp
typedef enum { Link, Thread } PointerTag;
// Link(0): 指向子节点, Thread(1): 指向前驱或后继节点

struct ThreadNode {
    int data;
    ThreadNode *lchild, *rchild;
    PointerTag ltag, rtag; // 线索标志位
};
```

- C++ 中序遍历

```cpp
ThreadNode* pre = NULL; // 记录中序遍历的前驱节点

// 中序线索化二叉树
void InThreading(ThreadNode* p) {
    if (p == NULL) return;

    // 递归线索化左子树
    InThreading(p->lchild);

    // 处理当前节点的前驱线索
    if (p->lchild == NULL) {
```

```
        p->lchild = pre;    // 左指针指向前驱
        p->ltag = Thread;   // 标记为线索
    }

    // 处理前驱节点的后继线索
    if (pre != NULL && pre->rchild == NULL) {
        pre->rchild = p;    // 前驱的右指针指向当前节点（后继）
        pre->rtag = Thread;
    }

    pre = p; // 更新前驱节点为当前节点

    // 递归线索化右子树
    InThreading(p->rchild);
}
```

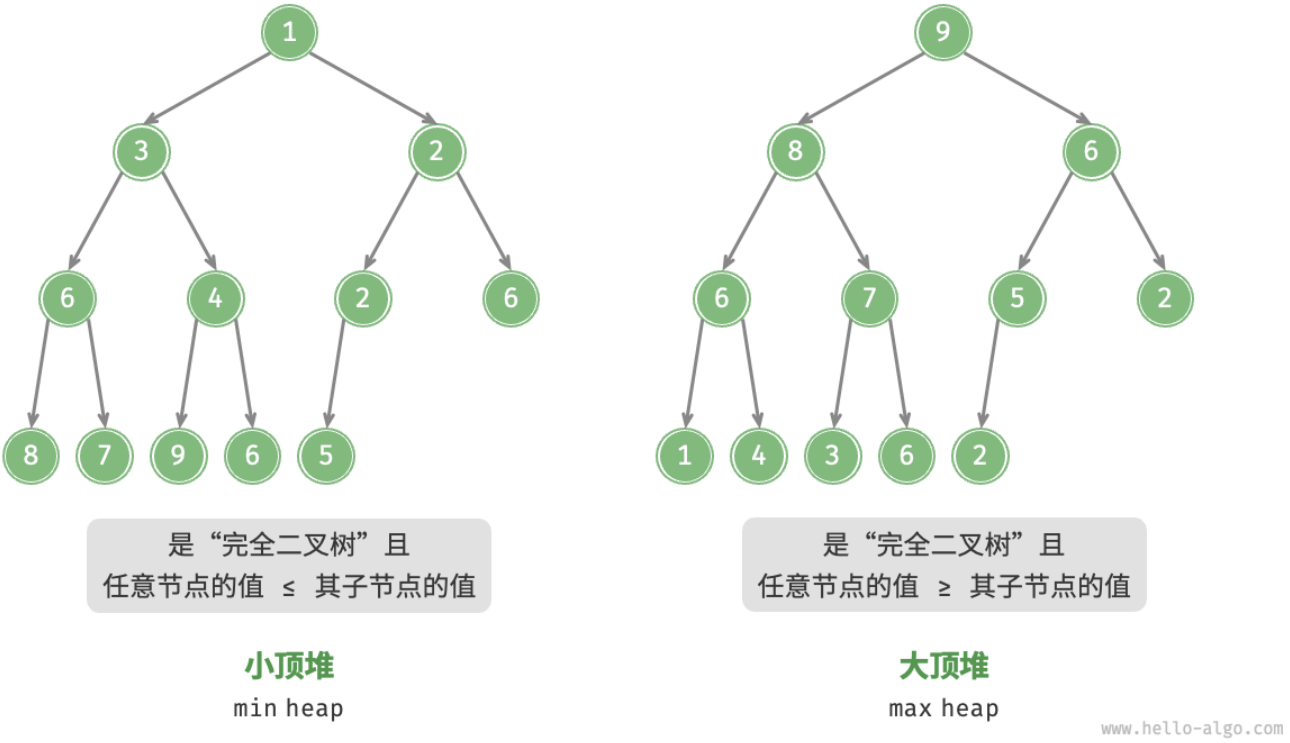
堆

> 快速找到最大值（或最小值）

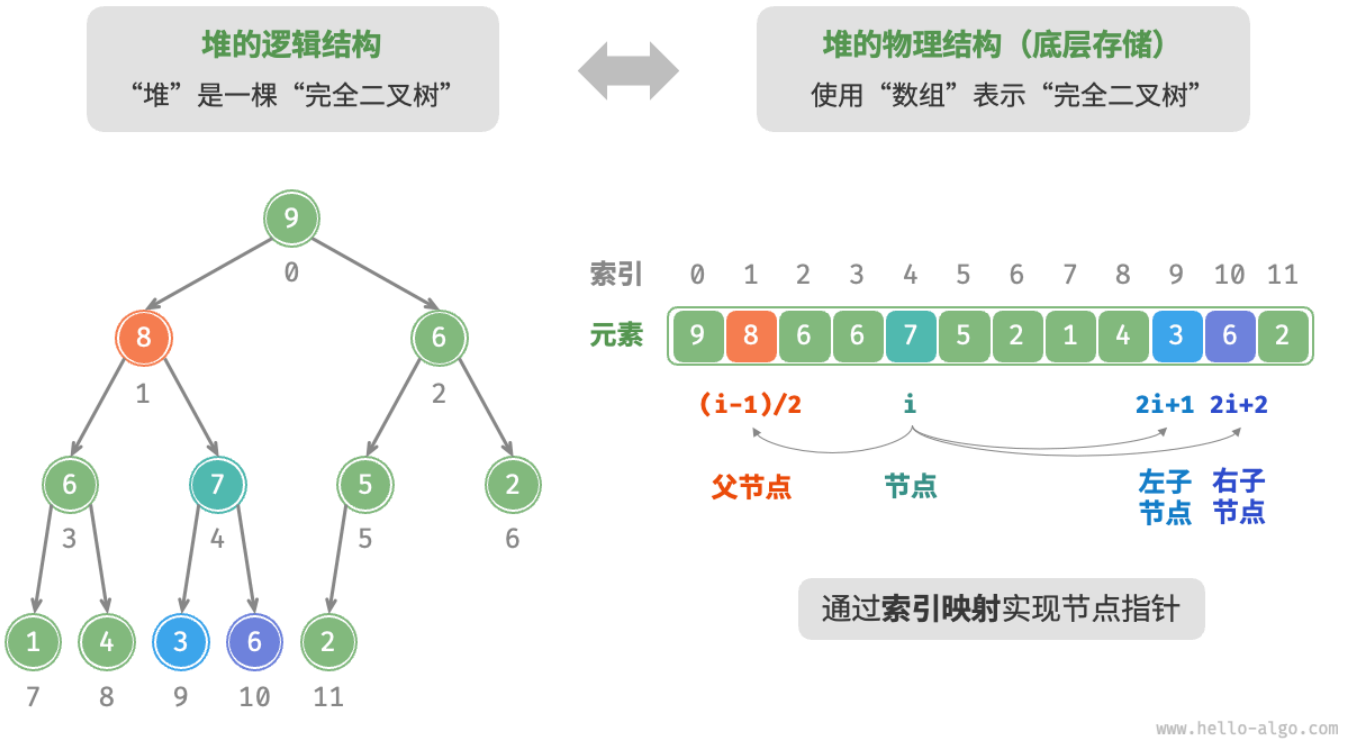
定义

满足以下两个条件：

- 结构条件： 它必须是一棵完全二叉树。
- 有序性：堆中每一个节点的值，都必须大于等于（或小于等于） 它的子节点。



> 堆的存储：数组



给定索引i，其左子节点的索引为$2i+1$，右子节点的索引为$2i+2$，父节点的索引为$(i-1)/2$（向下整除）。当索引越界时，表示空节点或节点不存在。

堆的分类

根据“父节点比子节点大”还是“小”，堆分为两类：

1. 大顶堆 / 最大堆

- 规则： 根节点是整棵树里最大的。任意节点的值 $\geq$ 它的子节点。

2. 小顶堆 / 最小堆

- 根节点是整棵树里最小的。任意节点的值 $\leq$ 它的子节点。

堆的核心操作

1. 插入元素——上浮

堆通常用数组实现完全二叉树，新元素需直接插入数组末尾（即完全二叉树的最后一个空位）。这一步的目的是避免完全二叉树出现 “空洞”，确保树的形态始终符合完全二叉树要求。

新元素插入后需与父节点比较，按堆的类型进行 “上浮” 调整：

- 若为大顶堆：若新元素值 > 父节点值，说明违反 “父节点值≥子节点值” 的堆序性，需交换新元素与父节点；
- 若为小顶堆：若新元素值 <父节点值，说明违反 “父节点值≤子节点值” 的堆序性，需交换新元素与父节点。

交换后，新元素继续与新的父节点重复上述比较，直到满足堆序性（新元素≤父节点 /≥父节点），或新元素成为堆顶（根节点）为止。

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// 大顶堆插入操作
```

```
void insertHeap(vector<int>& heap, int val) {
    // 1. 将新元素添加到堆末尾
    heap.push_back(val);

    // 2. 上浮调整：从新元素位置开始向上调整
    int index = heap.size() - 1;

    while (index > 0) {
        int parent = (index - 1) / 2;  // 计算父节点位置

        // 如果当前节点值大于父节点值，交换位置
        if (heap[index] > heap[parent]) {
            swap(heap[index], heap[parent]);
            index = parent;  // 继续向上检查
        } else {
            break;  // 堆性质已满足，停止调整
        }
    }
}

int main() {
    vector<int> maxHeap = {10, 7, 8, 5, 2};

    cout << "插入前: ";
    for (int x : maxHeap) cout << x << " ";
    cout << endl;

    insertHeap(maxHeap, 9);

    cout << "插入 9 后: ";
    for (int x : maxHeap) cout << x << " ";
    cout << endl;

    return 0;
}
```

- 时间复杂度：$O(\log N)$
- 空间复杂度：$O(1)$

2. 删除堆顶元素——下沉

堆顶是优先级最高的元素，删除时需先将堆的最后一个节点替换到堆顶位置 —— 这一步的目的是保证堆始终是完全二叉树，避免直接删除堆顶导致树结构断裂。

替换后的堆顶元素（原末位节点）通常不满足大顶堆 “父节点值≥子节点值” 的规则，需通过下沉调整验证其 “能力”：

- 比较该节点与其左右子节点，选出值最大的子节点；
- 若当前节点值小于这个最大子节点值，则交换两者位置；
- 重复上述过程，直到当前节点值≥所有子节点值，或该节点成为叶子节点。

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// 【核心】下沉操作
// index: 从哪个位置开始下沉（通常是 0）
// n: 堆的有效大小（因为是在数组里交换，不能越界）
void shiftDown(vector<int>& heap, int index, int n) {
    while (true) {
        int leftChild = 2 * index + 1;  // 左孩子下标
        int rightChild = 2 * index + 2; // 右孩子下标
        int maxIndex = index;           // 假设当前节点最大
```

```
        // 1. 看看左孩子是不是比我大
        if (leftChild < n && heap[leftChild] > heap[maxIndex]) {
            maxIndex = leftChild;
        }

        // 2. 看看右孩子是不是比“目前的赢家”还大
        if (rightChild < n && heap[rightChild] > heap[maxIndex]) {
            maxIndex = rightChild;
        }

        // 3. 决策时刻
        if (maxIndex != index) {
            // 如果最大者不是我自己，说明孩子比我强
            swap(heap[index], heap[maxIndex]); // 交换

            // 更新下标，准备在下一层继续 PK
            index = maxIndex;
        } else {
            // 我比两个孩子都大（或者没有孩子），位置稳了
            break;
        }
    }
}

// 删除堆顶函数
void popMax(vector<int>& heap) {
    if (heap.empty()) return;

    // 1. 把最后一个元素移到堆顶
    // (通常做法是直接 swap 头尾，然后删掉尾巴)
    swap(heap[0], heap.back());
    heap.pop_back(); // 物理删除原来的最大值

    // 2. 让新的堆顶下沉
    if (!heap.empty()) {
        shiftDown(heap, 0, heap.size());
    }
}

int main() {
    vector<int> maxHeap = {10, 8, 9, 5, 2};

    cout << "删除前: ";
    for(int x : maxHeap) cout << x << " ";
    cout << endl;

    popMax(maxHeap);

    cout << "删除堆顶(10)后: ";
    for(int x : maxHeap) cout << x << " ";
    // 预期输出: 9 8 2 5
    cout << endl;

    return 0;
}
```

哈夫曼树

定义

哈夫曼树是一种带权路径长度之和最小的二叉树（又称 “最优二叉树”）

1. 权重：节点的重要程度 / 出现频率
2. 路径长度：从根节点到某节点的边数；
3. 加权路径长度：单个节点的「权重 × 自身路径长度」；
4. 总加权路径长度：所有节点的加权路径长度之和。

> 权重越高的节点，其到根节点的路径长度越短

构造哈夫曼树

从给定的带权节点集合中，每次选出权重最小的两个节点，以这两个节点为左右子树，构建一个新节点（新节点的权重 = 两个子节点权重之和）；

然后将这两个原节点从集合中移除，把新节点加入集合；重复此过程，直到集合中只剩一个节点（即哈夫曼树的根节点）。

> 选两小，造新树，删两小，加新树

案例：

假设我们要给 4 个字母编码，它们的出现频率（权重）分别是： A: 2, B: 3, C: 6, D: 7

步骤 1：全员列队森林里有 4 棵小树苗：`{2, 3, 6, 7}`

步骤 2：选两小 (2, 3)

- 造新树： 把它俩合并，生成一个父节点，权重是它俩之和 $2 + 3 = \mathbf{5}$。
- 删两小，加新树： 从队列里划掉 2 和 3，加入 5。
- 当前队列： `{5, 6, 7}`

步骤 3：再选两小 (5, 6)：重复刚刚的操作

步骤 4：最后两小 (7, 11)：重复刚刚的操作，最后得到根节点为18

最终树的形状

```
          18 (根)
        /    \
       7      11 (5+6)
             /  \
            5    6 (C)
           / \
      (A) 2   3 (B)
```

哈夫曼编码

在计算机里，所有数据都是 0 和 1。

- 普通编码（定长）： 假设 A,B,C,D 都用 2 位表示：A=00, B=01, C=10, D=11。
- 哈夫曼编码（变长）：
  - 规定：向左走是 0，向右走是 1。
  - 根节点出发，走到叶子节点所经过的路径，就是该字符的编码。

C++ 代码

```cpp
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

// 哈夫曼树节点结构
struct Node {
    int weight;
    Node *left, *right;
    Node(int w) : weight(w), left(NULL), right(NULL) {}
};

// 优先队列比较函数：构建小顶堆
struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->weight > b->weight;  // 小顶堆使用大于号
    }
};

// 构建哈夫曼树
Node* buildHuffmanTree(vector<int>& weights) {
    // 创建小顶堆优先队列
    priority_queue<Node*, vector<Node*>, Compare> minHeap;

    // 将所有权重初始化为叶子节点
    for (int w : weights) {
        minHeap.push(new Node(w));
    }

    // 循环合并节点，直到只剩一棵树
    while (minHeap.size() > 1) {
        // 取出两个最小权重的节点
        Node* left = minHeap.top(); minHeap.pop();
        Node* right = minHeap.top(); minHeap.pop();

        // 创建新父节点，权重为子节点权重之和
        Node* parent = new Node(left->weight + right->weight);
        parent->left = left;
        parent->right = right;

        // 将新节点加入堆中
        minHeap.push(parent);
    }

    // 返回哈夫曼树的根节点
    return minHeap.top();
}

int main() {
    vector<int> weights = {2, 3, 6, 7};
    Node* root = buildHuffmanTree(weights);

    cout << "哈夫曼树构建完成，根节点权重: " << root->weight << endl;
    return 0;
}
```